

RAG y Agentes con LangGraph: Arquitectura de Sistemas de IA Generativa

Documento de referencia técnica en español — IA Generativa y Sistemas RAG

1. Introducción: Limitaciones de los LLMs sin RAG

Los Modelos de Lenguaje de Gran Escala (LLMs) como Gemini, GPT-4 o Claude son herramientas extraordinariamente capaces, pero presentan limitaciones fundamentales cuando se usan de forma aislada en aplicaciones empresariales:

Fecha de corte del conocimiento: Los LLMs tienen un conocimiento estático hasta la fecha de su entrenamiento. No conocen documentación interna de la empresa, normativas recientes, catálogos actualizados ni datos propietarios.

Alucinaciones: Cuando el modelo no tiene información suficiente, tiende a generar respuestas plausibles pero incorrectas con alta confianza aparente.

Contexto limitado: Aunque las ventanas de contexto han crecido significativamente, no es viable incluir toda la documentación relevante en cada consulta.

RAG (Retrieval-Augmented Generation) es la arquitectura que resuelve estas limitaciones combinando la capacidad generativa de los LLMs con la recuperación dinámica de información relevante desde una base de conocimiento propia.

2. Arquitectura RAG: Componentes y Flujo

2.1 Componentes principales

Componente	Función	Tecnología en este proyecto
Document Loader	Ingesta de documentos fuente (PDF, CSV, texto)	langchain DocumentLoaders
Text Splitter	Divide documentos en chunks manejables	RecursiveCharacterTextSplitter
Embedding Model	Convierte texto en vectores numéricos	Google Gemini Embeddings
Vector Store	Almacena e indexa los embeddings	ChromaDB (HNSW index)
Retriever	Recupera chunks relevantes por similitud	ChromaDB similarity search
LLM	Genera la respuesta final con el contexto	Google Gemini Pro
Memory	Mantiene el historial de conversación	LangChain ConversationMemory

2.2 Flujo de datos en un sistema RAG

Fase de indexación (offline, se ejecuta una vez):

- Cargar documentos fuente (PDFs, CSVs, textos)
- Dividir en chunks de tamaño controlado (ej: 500-1000 tokens con 100 de overlap)

3. Generar embeddings de cada chunk con el modelo de embeddings
4. Almacenar embeddings + texto original + metadatos en ChromaDB

Fase de consulta (online, en cada pregunta del usuario):

1. Recibir la pregunta del usuario
2. Generar embedding de la pregunta con el mismo modelo
3. Buscar los K chunks más similares en ChromaDB (por similitud coseno)
4. Construir el prompt: [system prompt] + [chunks recuperados] + [historial] + [pregunta]
5. Enviar el prompt al LLM y retornar la respuesta generada

2.3 Parámetros críticos del chunking

La estrategia de división de documentos en chunks tiene un impacto directo en la calidad de las respuestas. Los parámetros principales son:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter splitter = RecursiveCharacterTextSplitter( chunk_size=800, # Caracteres por chunk (aprox 200 tokens) chunk_overlap=100, # Solapamiento para no perder contexto en fronteras separators=['\n\n', '\n', '. ', ' ', ''], # Prioridad de separadores length_function=len ) # chunk_size pequeno (200-400): mejor precision, mas chunks, mas coste # chunk_size grande (800-1500): mejor contexto, menos chunks, posible ruido # Recomendacion para documentacion tecnica: 600-1000 caracteres
```

3. LangGraph: Agentes como Grafos de Estado

3.1 Concepto y motivación

LangGraph es una extensión de LangChain que permite construir agentes de IA como **grafos dirigidos de estado**. Cada nodo del grafo es una función que transforma el estado del agente, y las aristas definen el flujo de control (incluyendo bifurcaciones condicionales). Esto supera las limitaciones de las cadenas lineales (LangChain Chains) y permite implementar patrones como:

- Bucles de razonamiento (ReAct: Reason + Act)
- Decisiones condicionales basadas en la respuesta del LLM
- Llamadas a herramientas externas con manejo de errores
- Memoria persistente entre turnos de conversación

3.2 Estructura de un grafo LangGraph con RAG

```
from langgraph.graph import StateGraph, END from typing import TypedDict, List # Definir el estado del agente class AgentState(TypedDict): pregunta: str contexto_recuperado: List[str] historial: List[dict] respuesta: str # Nodo 1: Recuperar contexto relevante de ChromaDB def nodo_recuperacion(state: AgentState) -> AgentState: resultados = coleccion.query( query_texts=[state['pregunta']], n_results=4 ) state['contexto_recuperado'] = resultados['documents'][0] return state # Nodo 2: Generar respuesta con Gemini def nodo_generacion(state: AgentState) -> AgentState: contexto = '\n---\n'.join(state['contexto_recuperado']) prompt = f'Contexto:\n{contexto}\n\nPregunta: {state["pregunta"]}' respuesta = llm.invoke(prompt) state['respuesta'] = respuesta.content state['historial'].append({ 'pregunta': state['pregunta'], 'respuesta': state['respuesta'] }) return state # Construir y compilar el grafo grafo =
```

```
StateGraph(AgentState) grafo.add_node('recuperacion', nodo_recuperacion)
grafo.add_node('generacion', nodo_generacion)
grafo.set_entry_point('recuperacion') grafo.add_edge('recuperacion',
'generacion') grafo.add_edge('generacion', END) agente = grafo.compile()
```

4. Memoria de Conversación

La memoria de conversación es lo que transforma un sistema de pregunta-respuesta aislada en un verdadero asistente conversacional. Sin memoria, el agente trata cada pregunta como independiente y no puede responder referencias a intercambios anteriores como '¿y cuál de esas opciones recomendarías para mi caso?'.

4.1 Tipos de memoria en LangChain/LangGraph

Tipo	Descripción	Ventaja	Limitación
ConversationBufferMemory	Guarda todo el historial completo	Sin pérdida de información	Crece sin límite
ConversationBufferWindowMemory	Guarda los últimos K turnos	Coste de tokens controlado	Pierde contexto antiguo
ConversationSummaryMemory	Resume el historial con el LLM	Contexto compacto	Coste extra de inferencia
ConversationSummaryBufferMemory	Mezcla: buffer reciente + resumen antiguo	Balance óptimo	Más compleja de configurar

4.2 Implementación de memoria en el grafo

```
# El historial se pasa como parte del prompt en cada turno
def formatear_historial(historial: list, max_turnos: int = 5) -> str:
    if not historial: return ''
    # Tomar solo los ultimos max_turnos turnos
    ultimos = historial[-max_turnos:]
    lineas = []
    for turno in ultimos:
        lineas.append(f'Usuario: {turno["pregunta"]}')
        lineas.append(f'Asistente: {turno["respuesta"]}')
    return '\n'.join(lineas)

# Uso en el nodo de generacion
def nodo_generacion(state: AgentState) -> AgentState:
    contexto = '\n--\n'.join(state['contexto_recuperado'])
    historial_fmt = formatear_historial(state['historial'])
    prompt = ( f'Contexto de la base de conocimiento:\n{contexto}\n\n'
               f'Historial de conversacion:\n{historial_fmt}\n\n'
               f'Pregunta actual: {state["pregunta"]}' )
    respuesta = llm.invoke(prompt)
    state['respuesta'] = respuesta.content
    return state
```

5. Ingeniería del System Prompt

El system prompt define la identidad, el tono, las capacidades y las limitaciones del agente. Es uno de los factores más determinantes en la calidad de las respuestas. Un system prompt bien diseñado debe especificar:

- **Rol y dominio:** Quién es el agente y de qué sabe.
- **Tono y audiencia:** Cómo debe comunicarse (técnico, divulgativo, formal).
- **Comportamiento ante incertidumbre:** Qué hacer cuando no tiene información suficiente.
- **Restricciones:** Qué no debe hacer o responder.

```
SYSTEM_PROMPT = """ Eres un asistente experto en Data Science y Machine Learning,
especializado en algoritmos de recomendacion, optimizacion matematica y sistemas
de IA generativa con RAG. Responde SIEMPRE basandote en el contexto proporcionado
de la base de conocimiento. Si la informacion no esta en el contexto, indica
```

claramente que no dispones de esa informacion en tu base de conocimiento y sugiere donde podria encontrarse. Usa un tono tecnico pero claro. Incluye ejemplos de codigo Python cuando sea relevante. Estructura las respuestas largas con apartados. NO inventes datos, cifras o referencias bibliograficas. """

6. Evaluación de la Calidad del Sistema RAG

Evaluar un sistema RAG es más complejo que evaluar un modelo de clasificación tradicional, ya que intervienen dos componentes: la calidad de la recuperación y la calidad de la generación.

Métrica	Qué mide	Cómo calcularla
Faithfulness	Si la respuesta es fiel al contexto recuperado	LLM-as-judge o RAGAS framework
Answer Relevance	Si la respuesta responde la pregunta	Similitud coseno (pregunta vs respuesta)
Context Precision	Si los chunks recuperados son relevantes	Proporción de chunks útiles / total
Context Recall	Si se recuperaron todos los chunks necesarios	Requiere ground truth anotado

7. Referencias

- LangGraph Documentation: <https://langchain-ai.github.io/langgraph/> — Guía oficial de agentes con grafos de estado, incluyendo patrones de memoria y herramientas.
- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. Paper original que introduce la arquitectura RAG.
- RAGAS Framework: <https://docs.ragas.io> — Framework de evaluación automática de sistemas RAG con métricas como faithfulness y answer relevance.
- ChromaDB Documentation: <https://docs.trychroma.com> — Base de datos vectorial open-source, guía completa de indexación, consultas y configuración.